

Bonus task 5

Thick beam structure analyzed with two-dimensional four-node bilinear solid (plane) element

Possibility to earn six bonus points

Deadline: The 15th of June

Your task is to derive a two-dimensional quadrilateral four-node bilinear solid element (plane element "Q4") via MATLAB/Python symbolic toolbox, and use the derived element for solving displacements at the free end of a thick cantilever beam. Derive shape functions and find elemental stiffness matrix via strain energy based on a plane stress assumption. Code your FE-solution in MATLAB/Python using meshes 1×1 , 2×1 and $n_x \times n_y$ where $n_x \times n_y$ is the fine mesh for the converged solution. Use an equally distributed force at the free end of the beam. Solve nodal displacements at a free end and compare results to the analytical solution.

To solve the problem, use the given Matlab/Python templates:

<code>run_DeriveQ4_template_v11.m(py)</code>	run firstly to export shape functions and stiffness matrix
<code>run_plotShapef_template_v11.m(py)</code>	Approximate displacement field with given nodal displacements.
<code>run_LinStatic_Q4_BonusTask5_template_v11.m(py)</code>	Solves cantilever problem
<code>ElemDOFsConnectivity_Q4.m(py)</code>	Makes DOF's connectivity from nodal connectivity

Firstly, modify the `run_DeriveQ4_template_v11.m` (for MATLAB) or `run_DeriveQ4_template_v11.py` (for Python) file to derive the shape functions and the stiffness matrix.

These scripts generate:

- the **shape functions** in `ShapeFunctionsQ4.m` (MATLAB) or `ShapeFunctionsQ4.py` (Python), and
- the **element stiffness matrix** in `KlocQ4.m` (MATLAB) or `KlocQ4.py` (Python).

The shape functions are used to visualise the displacement field and compute the strain energy in

- `run_plotShapef_template_v11.m` (MATLAB) or
- `run_plotShapef_template_v11.py` (Python)

The stiffness matrix is then used in `run_LinStatic_Q4_BonusTask5_template_v11.m` (MATLAB) or `run_LinStatic_Q4_BonusTask5_template_v11.py` (Python) to solve the displacement at the free end of a thick cantilever problem using plane elements with a 2×1 mesh.

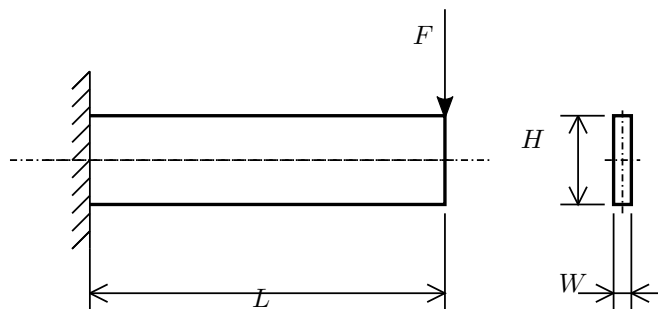


Figure 1. Thick cantilever problem

The parameters of the problem are given in Table.

Global nodal numbering

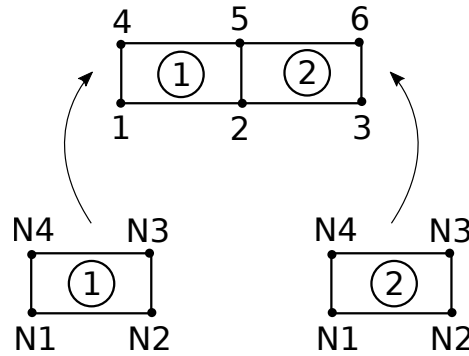


Figure 2. FE discretisation of the cantilever problem

Force, F [N]	Young's mod., E [Pa]	Poisson's ratio, ν	Length, L [m]	Height, H [m]	Width, W [m]
62500	$2.07 \cdot 10^{11}$	0.3	2	0.5	0.1

Write a small report that presents your results explicitly in a table. Submit the report in pdf format. Submit also all needed MATLAB/Python files (m-files) and check they run without problems. Please note that no points will be awarded without a report and MATLAB codes.

Please note that verifying displacement field approximation and strain energy element's size $L_x=2$ m, $L_y=2$ m and $L_z=0.1$ m need to be used. Bonus points will be awarded as follows:

- Correct displacement field approximation with derived shape functions (1 point). Modify and run `run_DeriveQ4_template_v10.m` or `run_DeriveQ4_template_v10.py` file to export shape functions, internal forces and stiffness matrix. You can test your displacement field approximation and internal forces function by modifying the `run_plotShapef_template_v11.m` or `run_plotShapef_template_v11.py` file. Give a 0.01 m displacement in the vertical direction at element's local nodes 2 and 3, and approximate the displacement field $u_h(x=0, y=0) = \mathbf{N}\mathbf{u}$ and $v_h(x=0, y=0) = \mathbf{N}\mathbf{v}$. Use element's size $L_x=2$ m, $L_y=2$ m and $L_z=0.1$ m in `run_plotShapef_template_v11.m` or `run_plotShapef_template_v11.py` file to check the displacement field.
- Correct strain energy (1 point). Finish strain energy form and export function for the strain energy with `run_DeriveQ4_template_v10.m` or `run_DeriveQ4_template_v10.py` file. Test the function providing a 0.01 m nodal displacement in the vertical direction at element's local nodes 2 and 3, and evaluate the vector of internal forces. Use element's size $L_x=2$ m, $L_y=2$ m and $L_z=0.1$ m in `run_plotShapef_template_v10.m` or `run_plotShapef_template_v10.py` file to check strain energy.
- Global assembling after the elimination of constraints (1 point). Compute the determinant and submit the result.
- Correct global nodal displacements $\{u_3, v_3, u_6, v_6\}$ at the free end with an element mesh 2×1 . Be careful with signs (1 point).
- Find converged global nodal displacements at the free end with an element mesh $n_x \times n_y$. Now you end a very fine mesh and you will have many elements also in the thickness direction, so it makes sense to take the average of displacements in u_x and u_y direction. Be careful with signs (1 point).
- Solve maximum displacement based on Timoshenko's beam theory (1 point) and compare the result with FE-solution.

Tips

Analytical solution based on Timoshenko's beam theory (linear theory)

$$\frac{d^2 v}{dx^2} = -\frac{M(x)}{EI(x)} - \frac{\kappa_{xy} q(x)}{GA} \quad (1)$$

Maximum deflection for the cantilever beam under applied force at the free end can be found as:

$$\delta_{max} = \underbrace{\frac{FL^3}{3EI}}_{\text{bending}} + \kappa_{xy} \underbrace{\frac{FL}{GA}}_{\text{shear}} \quad (2)$$

where κ_{xy} is a shear correction factor (needed to use in the beams based on the first-order shear deformation theory (FSDT)). It can be approximated to be 6/5 for a rectangular cross-section.

Derivation of shape functions

Displacement field interpolations:

$$\begin{cases} u_h(x, y) := a_0 + a_1 x + a_2 y + a_3 xy \\ v_h(x, y) := b_0 + b_1 x + b_2 y + b_3 xy \end{cases}.$$

Displacement interpolation is the same for both displacements $a \Rightarrow$ same shape functions. Four unknowns, four eqs:

$$\begin{cases} u_h(-l_x/2, -l_y/2) := a_0 - \frac{1}{2}a_1 l_x - \frac{1}{2}a_2 l_y + \frac{1}{4}a_3 l_x l_y = u_1 \\ u_h(l_x/2, -l_y/2) := a_0 + \frac{1}{2}a_1 l_x - \frac{1}{2}a_2 l_y - \frac{1}{4}a_3 l_x l_y = u_2 \\ u_h(l_x/2, l_y/2) := a_0 + \frac{1}{2}a_1 l_x + \frac{1}{2}a_2 l_y + \frac{1}{4}a_3 l_x l_y = u_3 \\ u_h(-l_x/2, l_y/2) := a_0 - \frac{1}{2}a_1 l_x + \frac{1}{2}a_2 l_y - \frac{1}{4}a_3 l_x l_y = u_4 \end{cases}.$$

By solving a_i 's:

$$\mathbf{a} = \mathbf{A}^{-1} \mathbf{u}_u \quad (3)$$

and substituting them into the Eq.(4):

$$u_h = \mathbf{a}^T \mathbf{p} = \mathbf{A}^{-T} \mathbf{u}_u^T \mathbf{p}^T = \underbrace{\mathbf{A}^{-T} \mathbf{p}}_{\mathbf{N}} \mathbf{u}_u \quad (4)$$

where $\mathbf{N}$ includes shape functions.

Please take a look at more details in the lecture slides.

Derivation of stiffness matrix

$$W_{int} = \frac{1}{2} \int_V \boldsymbol{\sigma} \boldsymbol{\varepsilon} dV = \frac{1}{2} \mathbf{u}^T \underbrace{l_z \int_x \int_y \mathbf{B}^T \mathbf{D} \mathbf{B} dy dx}_{\mathbf{K}} \mathbf{u} \quad (5)$$

$$\bar{\mathbf{K}} = \int_V \mathbf{B}^T \mathbf{D} \mathbf{B} dV = l_z \int_x \int_y \mathbf{B}^T \mathbf{D} \mathbf{B} dy dx = l_z \int_{-l_x}^{l_x} \int_{-l_y}^{l_y} \mathbf{B}^T \mathbf{D} \mathbf{B} dy dx \quad (6)$$

or more generally as:

$$\mathbf{f}_{int} = \frac{\partial W_{int}}{\partial \mathbf{u}} \quad ; \quad \bar{\mathbf{K}} = \frac{\partial \mathbf{f}_{int}}{\partial \mathbf{u}}$$

Assembling, BC's, applied forces and solution

You need to distribute force F for nodes at the free end. In static code, you need to uncomment lines and add forces with respect to correct DOF's.

```
%fglob(?)=?;           % add loading
%fglob(?)=?;           % add loading
```

Note that $\mathbf{K}_{G_{red}}$ is symmetric i.e. $k_{ij} = k_{ji}$ (if not, there is a typo) and $\det \mathbf{K}_{G_{red}} > 0$ (if not, check BC's).

And then we solve displacements (in a global coordinate system).

$$\mathbf{u}_{G_{red}} = \mathbf{K}_{G_{red}}^{-1} \mathbf{f}_{G_{red}}$$

In the case of a mesh of 2×1 , you need to add one more element in the nodal connectivity matrix:

```
ElemNodeConnectivity=[1,2,4,3];           % for one element. You need two lines for ...
two elements...
```

and then the element's DOF's connectivity is handled via ElemDOFsConnectivity_Q4.m function.

```
% Lets use functions that creates DOFs connectivity
ElemDOFsConnectivity=ElemDOFsConnectivity_Q4(ElemNodeConnectivity,NumberOfElems);
```

Note you need to change also lines

```
nxElems=1;                               % This you need to change. Discretization ...
    in x-direction
NumberOfNodes=4;
```